



Loan Affordability Agent API

Muhammad Umar Nadeem | Multi Agent Systems – Assignment 1 | 18th September 2026

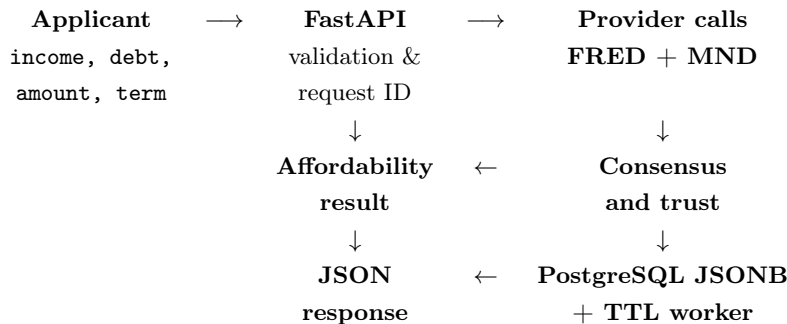
Page 1: System Architecture

Goal. The system answers the question: “Can an applicant reasonably afford the requested loan?” It exposes the endpoint `POST /v1/credit/affordability`. The request contains annual income, current monthly debt, requested loan amount, and loan term in months.

How the system works

1. **API request:** FastAPI validates the four input values and creates a request ID.
2. **Live data collection:** Two independent providers are queried concurrently for the current 30-year fixed mortgage rate:
 - FRED, using the `MORTGAGE30US` series;
 - Mortgage News Daily, using its daily rate index page.
3. **Consensus:** The two readings are checked for freshness and agreement. A freshness-weighted average becomes the consensus interest rate. The system calculates confidence, quality, and whether the result is verified.
4. **Affordability calculation:** The consensus rate is used in the standard loan amortization formula. The service calculates the monthly payment, estimated debt-to-income ratio (DTI), eligibility, and maximum affordable principal.
5. **Storage and retention:** PostgreSQL stores raw provider observations and consensus records as JSONB. A background worker creates daily rollups and removes records after their time-to-live (TTL).
6. **Response:** The API returns a standard JSON envelope containing `data` and `meta`. Metadata includes provenance, timestamps, freshness, trust metrics, latency, rate-limit information, and warnings.

Simple architecture view



Reliability and verification

Provider failures are reported as warnings; the service does not silently invent a replacement rate. If one provider fails, the available source can be used with reduced trust. If all sources fail, the API returns a structured error. The focused tests cover response shape, calculations, source failover, freshness, availability, and the mocked latency target.

During the live demonstration, the endpoint returned HTTP 200 with `eligible: true`, a consensus rate of 7.12%, both provider sources, `verified: true`, and no warnings. The demonstrated request used an income of 85,000, monthly debt of 400, requested amount of 250,000, and a 360-month term.

Page 2: Use of AI

AI tools and dates

Two AI tools supported this project, used for different parts of the workflow. **Claude (Anthropic, web chat interface)** was used for planning the system design, working through the consensus and TTL logic, generating and iterating on the FastAPI implementation, writing the test suite, and drafting setup documentation. **GitHub Copilot (Visual Studio Code)** was used locally for in-editor completion and small fixes while integrating the FRED key, running the service, and debugging environment-specific issues. Both tools were used between 16–18 September 2026. The exact underlying model variant for Copilot was not recorded in the project, so this report identifies it by tool name rather than a specific model.

Representative prompts and design exchanges

The following summarise the substantive prompts that shaped the implementation, rather than listing every message exchanged:

- Given the assignment’s five core requirements (multi-source consensus, standardized payload, PostgreSQL JSONB with TTL, and SLA testing) and the chosen task – *Loan affordability*, POST /v1/credit/affordability – requested a critical read of what “multi-source live consensus” should mean for a computed metric like affordability, since the output itself has no natural second source. This led to sourcing the live *interest rate* input from two independent providers instead, and computing affordability deterministically from the consensus rate.
- Asked for a comparison of realistic, freely-accessible provider options for a live 30-year fixed mortgage rate, weighing an official API (FRED) against a scraped industry page (Mortgage News Daily), including reliability trade-offs and how divergence between them should affect the confidence score.
- Requested the consensus algorithm be freshness-weighted rather than a flat average, and that source failure be handled as a distinct, typed error path (rate limit vs. unavailable vs. schema mismatch) so failures are never silently masked, per the assignment’s “no silent fallbacks” rule.
- Asked for the PostgreSQL JSONB schema and TTL worker to summarize expired micro-data into daily rollups before deletion, rather than simply discarding it, and for that logic to be idempotent and safe to re-run.
- Requested an SLA test suite covering availability under repeated polling, p95 latency, the freshness invariant (`age_seconds ≤ ttl_seconds`), and a source-failover scenario – fully mocked so it runs deterministically without a live database or network.
- Asked for the response payload to be checked field-by-field against the assignment’s canonical JSON schema, and for setup instructions (including why Docker was the simplest route for the PostgreSQL dependency) to be written in plain language for the README.

How AI assistance was used

AI assistance was used as design and implementation support: proposing the source/consensus strategy, generating the FastAPI service, schema, and worker code, and helping diagnose issues such as environment-variable loading, dependency compatibility, and differing payload shapes between the two providers. The student made the final decisions on task selection, source choice, and consensus design; supplied and kept the FRED credentials local; reviewed all generated code; and ran the project rather than treating AI output as proof that the system worked correctly.

Verification performed by the student

The implementation was checked by running the test suite, which completed with 6 passing tests. The live service was then checked through the FastAPI Swagger page at <http://127.0.0.1:8000/docs>. The health endpoint returned HTTP 200. The affordability request returned HTTP 200 and showed both FRED and Mortgage News Daily in the provenance list, `verified: true`, and an empty warnings list.

The FRED key remains in the local `.env` file and should not be submitted. The submitted project should contain only the safe example configuration, source code, schema, tests, README, and this report.